

ElasticDino: Dense Semantic Features using Elastic Deformation

Anonymous ECCV 2026 Submission

Paper ID #8289

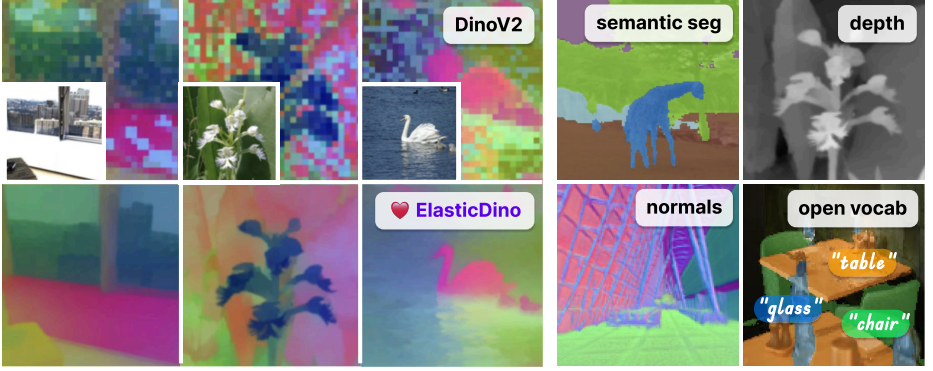


Fig. 1: ElasticDino upscales ViT features and corrects their artifacts, allowing to efficiently train a variety of dense vision tasks.

Abstract. Vision transformers (ViTs) are the backbone of countless vision tasks, including dense ones, such as segmentation and depth estimation. However, the quadratic complexity of self-attention limits semantic feature resolution and induces spatial artifacts. Dense prediction hence requires significant task-specific, computational and data collection efforts. In this work, we introduce **ElasticDino**, an adapter for these backbones, which produces high-resolution feature maps that preserve the fine details of the input image. As a key insight, we observe that all channels display consistent artifacts, which can be simultaneously resolved and matched to the input image structure. Using a lightweight, fully convolutional architecture predicting only spatial deformations, **ElasticDino** produces pixel-aligned, artifact-free, full-resolution feature maps. We choose deformations for their ability to efficiently affect channels consistently and strictly preserve sharp semantics, avoiding otherwise inherent smoothness biases. Given the lack of ground truth, we employ pixel-level linear heads to predict a set of downstream dense tasks, guiding the deformation process. **ElasticDino** enables efficient and high-performance training of dense tasks — we demonstrate superior performance to current methods, and further obtain state-of-the-art results on semantic segmentation over a range of popular benchmarks. We argue that **ElasticDino** is recommended for many of the dense tasks currently employing pretrained attention-based backbones.

1 Introduction

Foundational Visual features have become the backbone for most vision tasks in recent literature. Under this paradigm, a large foundational model is typically contrastively trained to embed images into a latent space. This embedding is rich and semantic by nature, and serves as a great starting point for various downstream tasks with significantly less training requirements. Specifically, self-supervised features trained using a Vision Transformer (ViT) architecture (i.e., DINO [3, 18, 24]) are employed in state-of-the-Art semantic, panoptic and instance segmentation [5, 15], object detection [17], monocular depth estimation [2, 22, 33, 34], optical flow estimation [26], and more.

Despite their stellar success, these models have an inherent flaw: built on self-attention, they require quadratic memory, imposing patchification of the input image, effectively restricting the resolution of the final feature maps and introducing artifacts in their semantics. Indeed, for most hardware, a patched resolution of 32×32 or 64×64 is typically used, even when training dense tasks such as segmentation or depth estimation. Typically, a densification head is added to recover full resolution, (e.g. DPT [21]), which requires large amounts of data to recover fine details, as attested by the large and complex datasets used to train monocular depth estimation tasks [33, 34]. To make matters worse, this requires training a specific head for each downstream task due to poor transferability.

In this paper, we introduce *ElasticDino*, a lightweight deformation-based super-resolution approach to enhance DINO features, facilitating the training of lightweight heads for individual downstream tasks with fewer data and a fraction of the computation. As we demonstrate, current approaches following a similar path base their upscaling process on reconciling high and low resolution features through cropping-like mechanisms, leading to inevitable bleeding or drifting of feature semantics at boundaries, which we strictly mitigate through two key observations.

First, We observe that due to patchification, artifacts are consistent across feature-channels, and can thus be resolved for all channels simultaneously (see Figure 3), taking into account only their structure and not their actual values. As such, matching of feature and image structure is the only necessary task. Second, guided by the original image, we learn deformation fields that align all feature channels to the image structure, alternating between upsampling and deformation. We observe that deformations offer efficient structure preservation, inherently and consistently adhering to sharp semantics. This alleviates otherwise inherent smoothness biases that are typical to the mechanisms used by previous work, and hence minimizes bleeding and fuzzy transition artifacts. Critically, since no upscaled ground truth exists in our setting, we train **ElasticDino** using a set of five lightweight fully-connected downstream heads, each learning to predict a different dense map, while limited to a 1×1 receptive field, forcing pixel-level matching of feature semantics. Again, this alleviates the need for cropping or linearly changing image resolutions and further encourages sharp semantics in the resulting features.

Through careful evaluation, we demonstrate that **ElasticDino** produces superb alignment of semantics and structure, while removing other spurious artifacts such as those induced by positional encoding. Using lightweight heads, we report clearly superior results compared to other upscaling methods on semantic segmentation, open-vocabulary segmentation, and depth estimation, with minimal computational effort. We furthermore demonstrate superior results on semantic segmentation benchmarks, compared to leading task specific art.

We argue **ElasticDino** is recommended for most dense tasks currently employing pre-trained ViT backbones, incurring negligible computational costs while introducing sharper features with less artifacts.

2 Related work

2.1 Vision transformers and semantic features

Shortly after its inception for text data [27], the Transformer architecture was adapted to vision tasks by viewing an image as a sequence of tokens, yielding Vision Transformers [11]. Unfortunately, due to the quadratic complexity of the attention mechanism, feeding images as a sequence of pixels is prohibitively costly beyond resolutions larger than 32x32 or 64x64. Therefore, the standard method for tokenizing an image, following the seminal paper, is to decompose the image into a sequence of non-overlapping patches, usually leading to a 16x16 grid.

Vision transformers have shown exceptional performance on vision tasks, being able to model complex long-range dependencies, and setting state-of-the-art results in a variety of vision tasks such as classification [36], segmentation [5, 15], object detection [17], monocular depth estimation, [2, 33, 34], joint text-image encoding [20], Vision Large Language Models [29], and image generation [19].

Of note, many of the above tasks relied on pre-trained ViT encoders, much like vision models previously relied on convolutional backbones such as ResNet [13]. Indeed, ViTs have shown the ability to generate spatial semantic features, which transfer to a variety of tasks. Chief among these are Dino [3] and DinoV2 [18], which through unsupervised training provide exceptionally strong semantic features - DinoV2 has since become the go-to backbone for various state-of-the-art models [17, 33].

2.2 Correction of ViT artifacts

In order to correct for artifacts in ViT (in particular DINO) activations, several methods have been introduced - though without considering higher feature resolutions.

Soon after the introduction of DINO, it was realized that a large part of the artifacts encountered were due to a leakage of global semantics in patch features [9]. "Register" tokens were introduced and added to the tokenized image

sequence, meant to capture global semantics, while letting patch features absorb relatively more local semantics. Nowadays, the "with registers" versions of DINO and DINOv2 are almost exclusively used rather than the originals (including in this work).

Self-Patch [35] improves upon DINO's self-supervision approach by enforcing local invariances among neighboring patches, improving spatial consistency and downstream performance although without upsampling features to higher resolutions. This however requires retraining of the model.

More recently, **Denoising Vision Transformers** [32] was introduced to mainly resolve the artifacts introduced by positional encodings. They achieve this through decomposing feature maps into a "ground truth" component and several shared positional noise components. They impose consistency between the features produced by image crops, thus removing positional information from the image features. It is important to note that positional information in feature maps is nonetheless important; removing it entirely severely reduces the out-of-the-box ability to model positional dependencies between objects or points of the image.

2.3 ViTs and dense vision tasks

Surprisingly, few methods have been developed to increase the resolution of ViT features.

Dense Prediction Transformer (DPT) [21] has become the reigning architecture for ViT densification. It combines a ViT backbone with a convolutional decoder to generate high-resolution depth maps, and has led to great success in tasks such as monocular depth estimation [22, 33, 34]. **ViT-Adapter** [4] adds a trainable module to a ViT, injecting vision-specific priors. While displaying impressive performance, DPT exhibits difficulty reproducing high-frequency details, and above-all demands data-intensive per-task training, thus precluding their use as a task-agnostic backbone and restricting their applicability to specialized domains.

FeatUp [12] introduces two separate upscaling methods, both based on reconciling features for various augmentations of an image. The first is trained per-image and demonstrates honorable performance, but requires over a minute of training for every image, precluding most real-world applications. The second method results in a generic upscaler, which displays gross artifacts - unavoidable with the joint-bilateral-upsampling kernels employed.

Similarly, **Lift** [25] uses a simple CNN architecture which combines image and features and then upsamples them. The training procedure consists in matching upscaled features produced at a lower image resolution to features produced at higher image resolution, and not upscaled. As the chosen "ground truth" is not free of artifacts, the incentive to fully resolve them is not complete. Further, the editing of feature values, which are mixed with image encodings, does not guarantee strict preservation of semantics.

LoftUp [14] passes images, features and positional encodings to a of cross-attention blocks. The training procedure is two-staged. First, they compute vari-

ous segmentation masks on the image, use a form of bilateral upsampling guided by the masks, and use these as pseudo-labels. In a second stage, they perform teacher-student distillation, now using as objective the reconciliation of feature values along crops. This suffers the exact same issues as FeatUp and Lift, with direct feature value editing, and reduced incentive to correct artifacts, especially bleeding.

JAFAR [8] uses cross-attention using the encoded image as the query, features as value, and both as key, to compute pixel-to-semantics attention scores and query them within the feature values. This ensures that feature semantics are preserved. The training procedure is identical to LIFT and as such lacks the incentive to fully resolve artifacts.

AnyUp [30] interestingly proposes a method which is backbone-agnostic and transfers without retraining of the upscaling head. The architecture is essentially the same as for JAFAR, with an additional feature-agnostic layer which allows transfer. The training procedure is similar to FeatUp, using only crops as augmentations. Again, this does not provide enough incentive to resolve artifacts.

3 Method

ElasticDino is a super-resolution network for DINO features, learned as iterative upsampling followed by elastic deformations of the features. Given an image and the corresponding feature maps generated by the ViT, our goal is to upscale the feature maps while matching the image structure. Figure 4 depicts an overview of the architecture: The input features are repeatedly upscaled and deformed, with the deformation field being predicted using convolutional layers. We note that the advantages of this approach are lightweight computation that allow for higher-resolution maps, and better handling of high-frequency edges. Another major advantage of this approach is that since we sample from the original feature map, we diminish hallucination hazards, ensuring strong structural and semantic preservation.

As there are no ground truth upscaled features available, we train 1×1 convolutional heads on a collection of dense downstream tasks, ensuring no spatial context and pixel-wise feature and structure matching.

3.1 Preliminaries

Because of their patch-wise decomposition of the image and combined use of positional encodings, ViTs introduce significant artifacts in feature maps. We identify two main types of artifacts (Fig 2). First, in regions of near-constant color, positional artifacts are prominent and values may vary widely, without semantic difference. It has been observed in [9] that this happens in great part from global semantics being locally encoded - they alleviate this issue but artifacts remain. We explain this by the use of positional encodings, as they are the only varying input to the ViT in a constant patch. Second is feature bleeding (Figure 2 bottom): objects and boundaries are dilated due to discretization.

We make the crucial observation that ViTs introduce artifacts in a way which is consistent across channels (Fig. 3). This means that one can learn a deformation and apply it to all channels simultaneously, leading to very efficient resolution.

3.2 Architecture

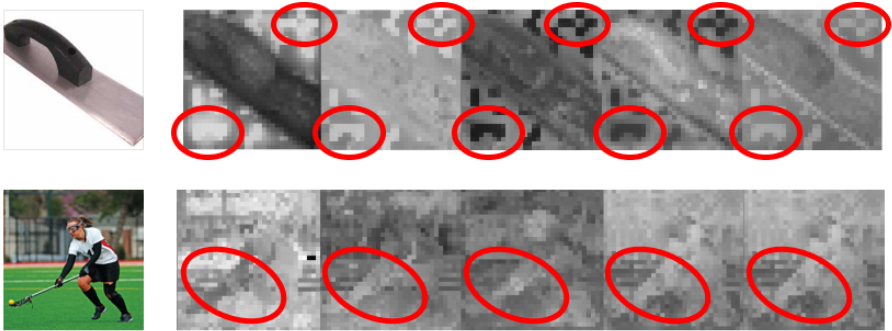


Fig. 2: Each of the grayscale images displays feature values for a select feature map channel. Artifacts introduced by vision transformers are seen to be consistent cross-channel. On the top row, regions of constant color display positional artifacts, and on the bottom row, we observe typical thickening of object edges (here the lacrosse stick).

The architecture, depicted in Fig. 4 consists of alternating Feature Deformation layers, which correct artifacts and 2x bilinear upsamples. A first deformation layer is applied, then followed by an alternation of upsampling and deformations. The deformation process is guided by the input image, allowing to introduce the image structure at that resolution to guide the deformation of the features.

Each upscaling stage is composed of a series of Feature Deformation Blocks, which take as inputs the dirty features at the current resolution and the image at the same resolution. These are projected to a common hidden dimension and concatenated; and then fed to a depthwise-separable [6] residual convolution network, which outputs a 2D deformation field. The field is applied to the input features to recover the denoised features.

In other terms, given features f , an image I at a current identical resolution, a single deformation block consists of two projections p_f , p_I , a resnet encoder R which outputs a 2D deformation field

$$F = R(p_f(f) \oplus p_I(I))$$

Which is applied coordinate-wise:

$$\hat{f}_{u,v} = f_{F_{u,v}}$$

The motivation for using depthwise separable convolutions is the desire to achieve an extremely lightweight architecture, given the fact that feature dimensions are already high, which at higher resolution leads to heavy memory and compute utilization. We observed no performance degradation as well as much faster training and inference.

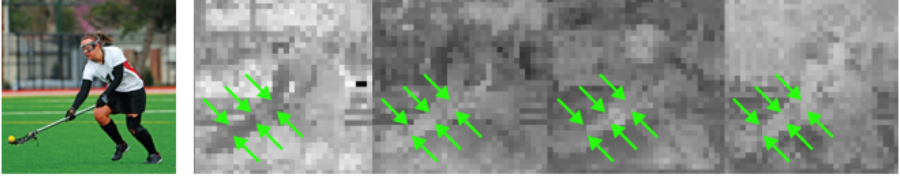


Fig. 3: Because artifacts are consistent across channels, applying the same deformation to all channels will mitigate them consistently.

Pixel-wise downstream tasks for semantic guidance Unfortunately, we have no ground truth maps in the upscaled resolution, and thus need to define an alternate training objective.

We make the almost tautological observation that a well-upscaled and de-noised feature map should be able, by design, to predict any dense task pixel-wise.

We hence introduce a multi-task learning approach, where we simultaneously train linear heads for downstream dense vision tasks (see Section 3.3). Crucially, the 1×1 receptive field prevents the heads from combining feature values from different spatial positions, forcing the predictions to only use the semantics at that exact pixel. Should a larger receptive field be employed, the incentive to match the semantics at the current pixel would be removed, and structure would be learnt both in the upscaler and the downstream head.

It is important to note that this approach can only work because we are employing deformations and sampling from the features themselves. Indeed, if the upsampling process were to directly edit feature values, the semantics of the downstream tasks could easily just become "baked in" to the feature maps, without any incentive to jointly edit all channels simultaneously. We show the effect of such an approach in an ablation study- Fig. 6.

For each downstream task (see below), we employ a simple linear head to predict the target. Furthermore, to encourage faster learning at every resolution, the same head is applied to every intermediate feature map, and the corresponding multi-resolution predictions are all used to compute the training objective.

3.3 Losses and Tasks

All the tasks we use predict a dense map on the full resolution. For every task except segmentation (for which we use the standard cross-entropy loss), the base

loss is defined as $\mathcal{L}_{\text{base}}(x, \hat{x}) = \|x - \hat{x}\|^2 + \text{SSIM}(x, \hat{x})$, where SSIM denotes the structural similarity loss [28]. For all tasks except segmentation, and in order to encourage reconstruction of high frequency details, we additionally penalize discrepancies between the channel-wise spatial gradients and laplacians, following [2]: $\mathcal{L}_{\text{grad}}(x, \hat{x}) = \mathcal{L}_{\text{base}}(\nabla x, \nabla \hat{x})$, $\mathcal{L}_{\text{lap}}(x, \hat{x}) = \mathcal{L}_{\text{base}}(\Delta x, \Delta \hat{x})$

To encourage faster reconstruction at every stage, we compute the loss for all intermediary feature maps, as well as at every resolution. Therefore, denoting $\hat{x}_{i,j}$ the output of the i -th stage for the j -th prediction head, $\hat{x}_j$ the ground truth for task j , $r_k = 32 \times 2^k$ where $k \in [[0, 3]]$ the resolutions used, and $\mathcal{L}_j$ the loss for task j (see the task list below), we apply the following multi-scale loss:

$$\mathcal{L}_{\text{ms}}(x, \hat{x}) = \sum_{i,j,k} \mathcal{L}_j(\text{interpolate}(x_i, r_k), \text{interpolate}(\hat{x}_{i,j}, r_k))$$

This allows gradients to flow at each stage, rather than through the entire upscaling network, greatly stabilizing and speeding up the learning.

The downstream tasks we use represent a diverse sets of maps to be predicted from the image. We find each to help improve overall embedding quality (see Section 4), and believe more can be added. We use the following downstream tasks: RGB reconstruction, monocular depth, semantic segmentation, surface normals, intrinsic decomposition. Training is performed on two interspersed datasets: Hypersim [23] and Imagenet-22k [10]. See Supplementary material for details on losses and the training procedure.

4 Experiments

We perform quantitative and qualitative evaluation on a prototype designed to upscale DinoV2 features, and report the results below. We report semantic segmentation and depth estimation comparisons, and open vocabulary segmentation in the supplementary material. Following recent publications [30], we also perform some of the same experiments for DinoV3 [24] features. For implementation details, including results from our training head tasks, see supplementary material.

Qualitative Results are presented in Figure 5. The upscaled features themselves are depicted in the top left. We visualize the top-3 PCA components of each feature map. As can be seen, artifacts are corrected and crisp semantic edges are formed that fit nicely to the original image structure. This includes separation of far and near objects (e.g., the bars of the vehicles cabin), and of appearance from geometry (e.g., the symbol in the shield). The upscaled features are compared to their original low-res counterparts and to previous approaches. For a more comprehensive demonstration of qualitative results, please refer to the supplementary material.

4.1 Benchmarks on downstream tasks

Semantic segmentation We train simple convolution heads for semantic segmentation on top of **ElasticDino** and compare performance with two leading

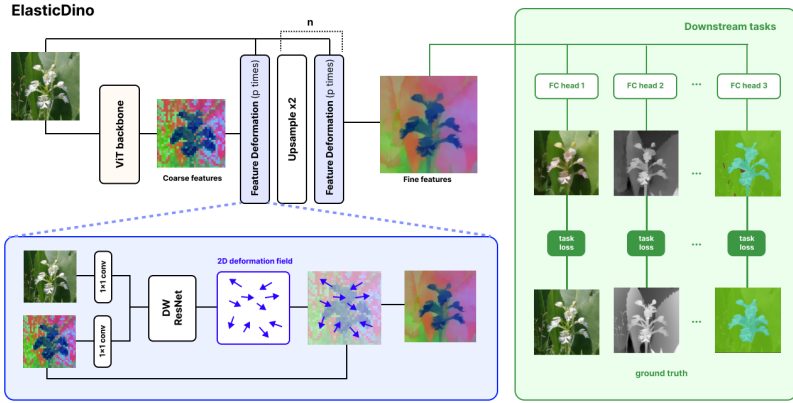


Fig. 4: Method overview. The input image is passed to the (Dino) backbone for the low resolution, artifact-full features. These are successively passed to a sequence feature-fixing layers and then bilinearly upsampled by a factor 2. **Feature Deformation Layers** project and concatenate both the image and the semantic features, which are fed to a series of depthwise residual blocks, that end up generating a deformation field that warps the feature map. **Task heads** induce the sole losses of the system. They are simple MLPs that operate on the individual pixel level. Each head is compared to the ground truth using a task-specific loss.

dedicated segmentation methods, Mask2Former [5] and OneFormer [15]. The data sets examined are MS-COCO [16], ADE20k [37, 38] and Cityscapes [7] (Cscapes), all standard data sets in semantic segmentation benchmarks.

ElasticDino displays higher performance as measured by Mean-IoU (Intersection over Union) on all three validation datasets, with a very large margin on ADE20k and Cityscapes. Qualitatively (In Supplementary material) we observe not only better classification ability, but much finer segmentation boundaries. This implies obvious advantages to downstream applications where sensitivity is key, such as autonomous driving.

Feature upsampling As our primary comparison, we further compare to task-agnostic methods that employ Dino feature upsampling (Table 3). These include training free Nearest neighbor and bilinear upsampling, and four state-of-the-art methods [8, 12, 14, 30]. We follow established evaluation protocols, as described in by JAFAR [8], measuring mean Intersection over Union (mIoU) and pixel-level accuracy (Acc) for the semantic segmentation task over four datasets, and Root Mean Square Error (RMSE) and δ_1 (measuring the percentage of pixels where that are less than 25% of the ground-truth) for the depth estimation task over one dataset. In the supplementary material, we also compare performance for zero-shot open vocabulary segmentation. As can be seen, **ElasticDino** demonstrates leading performance consistently for almost all dataset. Specifically, **ElasticDino** shines at small scale features and reduce bleeding between regions, even

when these do not align with RGB color differences. Results consistently show eliminated background artifacts for constant color background (stemming from positional encoding, e.g. the field in the eagle’s background), constant semantic in the presence of different colors (e.g., the flag in the background) and different semantics for similar colored pixels when necessary (e.g., the purple tap). Notably, these improved results are produced with less computational efforts ($\times 1.7 - \times 2.4$ better) and memory footprint ($\times 1.7 - \times 9$ better), as reported in Table 1.

Table 1: Run time comparisons of **ElasticDino** with competing feature upscaling works over DinoV2

	ED (ours)	AnyUp	LoftUp	JAFAR
Avg Time (fw pass)	186.32 ms	762.82 ms	747.29 ms	318.05 ms
Peak mem (fw pass)	923.01 MB	1652.09 MB	8310.90 MB	7337.48 MB
#Params	27,095K	878K	4,303K	628K

Finally, following AnyUp [30], we also perform a similar evaluation over the DinoV3 [24] backbone in Table 4, showcasing similar advantages.

Table 2: Validation Mean IoU for semantic segmentation benchmarks. Note that results differ from the comparison benchmarks, as we use a convolutional head (rather than linear) and train for longer than specified in the JAFAR benchmark.

Model	Cscapes	COCO	ADE20k
ElasticDino-L	0.6875	0.6270	0.5299
Oneformer-sw-L	<u>0.6412</u>	<u>0.6264</u>	<u>0.5109</u>
Mask2Former-sw-L	0.5986	0.6239	0.4928

4.2 Ablation studies

We perform an ablation study on the architecture and design choices. In order to ensure a fair benchmark, we train ElasticDino and its ablations on the Hypersim dataset and omit the depth task, reserving it for evaluation. The tasks are prediction of RGB, illumination, reflectance, residuals and normals, as defined in the paper by [23]. First, we ablate the deformations, employing convolutions only. Second, we ablate tasks in a leave-one-out fashion. For each trained instance, we train a UNet head to predict depth on Hypersim, reserving a fraction as a validation with SiLogLoss as the metric. Quantitative results are summarized in table 5, and some qualitative results are shown in figure 6.

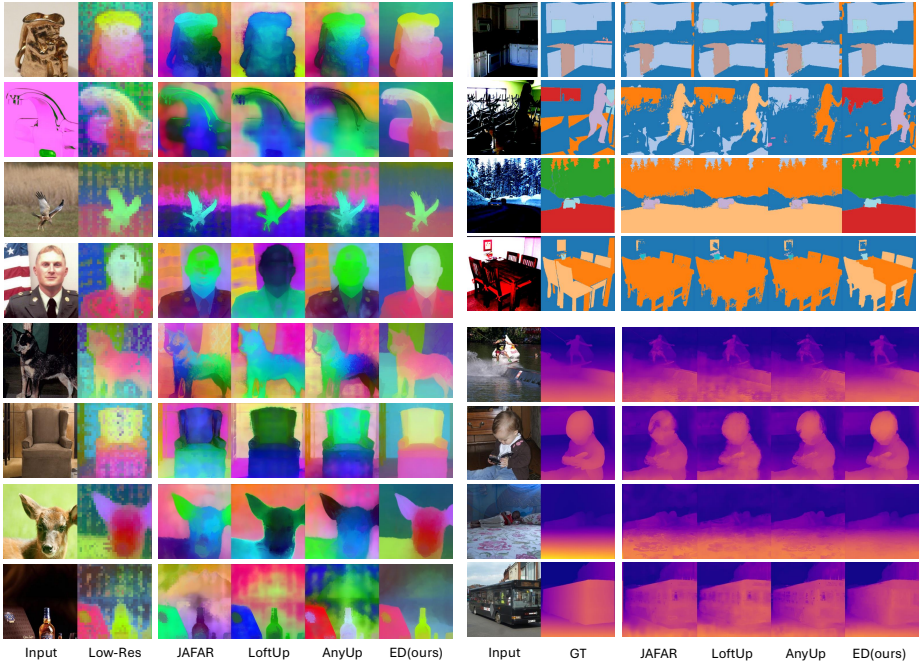


Fig. 5: Qualitative evaluation. Left: The top three PCA components of the feature maps of input images are depicted, as generated originally by DinoV2 (Low-Res), three previous approaches, and us. Top Right: Semantic segmentation maps of input images, as generated by humans (GT), three previous methods, and us are also depicted. Bottom right: depth maps for the same approaches and input setting are depicted as well. Throughout all results, **ElasticDino** depicts sharper edges that respect the input image resolution and mitigates semantic bleeding between regions, even when RGB colors are similar.

Table 3: Comparison of various upsampling methods. All benchmarks are run using the method and code used for the JAFAR paper. Bold = best, underline = second best.

Method	Semantic Segmentation								Depth Est.	
	COCO		VOC		ADE20K		Cityscapes		COCO	
	mIoU↑	Acc↑	mIoU↑	Acc↑	mIoU↑	Acc↑	mIoU↑	Acc↑	δ_1 ↑	RMSE↓
Nearest	56.17	76.97	76.41	93.80	37.27	71.91	54.05	90.36	58.08	0.70
Bilinear	59.03	79.07	80.70	95.17	39.23	73.69	59.37	92.47	59.92	0.66
FeatUp [12]	60.10	79.95	81.08	95.32	38.82	73.74	56.06	91.86	<u>61.69</u>	<u>0.64</u>
LoftUp [14]	<u>61.62</u>	<u>81.11</u>	84.18	<u>96.22</u>	<u>41.51</u>	<u>75.41</u>	62.13	93.53	58.01	0.68
JAFAR [8]	60.32	80.30	<u>84.19</u>	96.18	40.16	74.39	<u>61.58</u>	<u>93.45</u>	60.05	0.65
AnyUp [30]	60.68	80.59	83.81	96.13	40.39	74.76	60.99	93.37	60.65	<u>0.64</u>
ED (ours)	62.72	81.74	86.00	96.78	42.10	76.00	61.33	93.44	64.14	0.57

Table 4: Comparison of AnyUp and ElasticDino over DinoV3-small backbone. Benchmarks are run using the method and code used for the JAFAR paper. Bold = best.

DinoV3	Semantic Segmentation								Depth Est.	
	COCO		VOC		ADE20K		Cityscapes		COCO	
	mIoU(↑)	Acc(↑)	mIoU(↑)	Acc(↑)	mIoU(↑)	Acc(↑)	mIoU(↑)	Acc(↑)	δ_1 (↑)	RMSE(↓)
AnyUp	30.03	57.88	23.66	76.46	13.00	53.99	19.52	81.18	54.81	0.77
ED (ours)	64.04	82.43	86.26	96.89	44.57	77.03	62.91	94.03	61.10	0.62

The ablation of deformations results in almost imperceptible differences in feature values: because of the lack of incentive for the network to jointly modify features, task semantics are baked into the feature values. However, because some tasks exhibit correlation with depth prediction (e.g. surface normals), and because residual connections let the features flow through, some ability to transfer is conserved. When ablating each task individually, we see that every task leads to some amount of improvement in performance, showing that adding more tasks improves the performance of **ElasticDino**.

5 Discussion

5.1 Limitations

While demonstrating promising performance, **ElasticDino** struggles with distinguishing small holes in the source image that are smaller than patch size (see inset figure). Further investigation is required to determine whether the frequency at which this occurs is an intrinsic limitation of our approach, or if further training, a different architecture or better data could alleviate it.



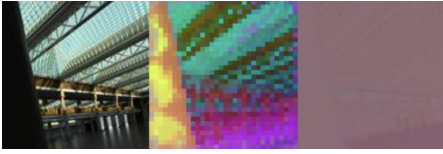


Fig. 6: Ablating deformations removes the incentive of the model to preserve original features and only bakes the task values directly into the output, with only imperceptible variations in feature values.

Model	Validation loss
No deformations	0.4055
w/o RGB	0.3564
w/o diffuse illumination	0.3544
w/o diffuse reflectance	0.3566
w/o residuals	0.3552
w/o normals	0.3482
ElasticDino	0.3478

Table 5: Ablation study using depth prediction on the Hypersim validation subset. We report the SiLog training loss, and ablate using traditional convolutions instead of deformation predictions and the five dense tasks used during training.

5.2 Future directions

Beyond solving the above-mentioned issue, a clear future direction is the extension of ElasticDino to other modalities, such as video or 3D. Video could be handled through video-specific feature generators such as VJEPa-2 [1]. 3D could be handled through multi-view projection of features from renderings, as done in TRELLIS [31] - it could itself improve TRELLIS by better aligning features and latent voxels.

5.3 Conclusion

We have introduced **ElasticDino**, a solution for producing arbitrary resolution pixel-wise semantic features on top of any vision transformer. We demonstrated that **ElasticDino** achieve clearly superior performance compared to existing methods, and state-of-the-art results on the specific downstream task of semantic segmentation.

ElasticDino, like the vision transformers it enhances, does not produce content that can be directly misused. As a foundational model, any broader impact, positive or negative, is likely to stem from improved performance in downstream tasks or the emergence of new applications.

We refer to the supplementary material for additional implementation details and qualitative results.

367 [1] Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Mojtaba, Komeili, 367
368 Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., 368
369 Gejji, A., Martin, A., Hogan, F.R., Dugas, D., Bojanowski, P., Khalidov, 369
370 V., Labatut, P., Massa, F., Szafraniec, M., Krishnakumar, K., Li, Y., Ma, 370
371 X., Chandar, S., Meier, F., LeCun, Y., Rabbat, M., Ballas, N.: V-jepa 2: 371
372 Self-supervised video models enable understanding, prediction and planning 372
373 (2025), <https://arxiv.org/abs/2506.09985> 373
374 [2] Bochkovskii, A., Delaunoy, A., Germain, H., Santos, M., Zhou, Y., Richter, 374
375 S.R., Koltun, V.: Depth pro: Sharp monocular metric depth in less than a 375
376 second (2025), <https://arxiv.org/abs/2410.02073> 376
377 [3] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., 377
378 Joulin, A.: Emerging properties in self-supervised vision transformers. In: 378
379 Proceedings of the IEEE/CVF international conference on computer vision. 379
380 pp. 9650–9660 (2021) 380
381 [4] Chen, Z., Duan, Y., Wang, W., He, J., Lu, T., Dai, J., Qiao, Y.: Vision 381
382 transformer adapter for dense predictions (2023), [https://arxiv.org/](https://arxiv.org/abs/2205.08534) 382
383 [abs/2205.08534](https://arxiv.org/abs/2205.08534) 383
384 [5] Cheng, B., Misra, I., Schwing, A.G., Kirillov, A., Girdhar, R.: Masked- 384
385 attention mask transformer for universal image segmentation. arXiv (2021) 385
386 [6] Chollet, F.: Xception: Deep learning with depthwise separable convolutions 386
387 (2017), <https://arxiv.org/abs/1610.02357> 387
388 [7] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, 388
389 R., Franke, U., Roth, S., Schiele, B.: The cityscapes dataset for semantic 389
390 urban scene understanding (2016), <https://arxiv.org/abs/1604.01685> 390
391 [8] Couairon, P., Chambon, L., Serrano, L., Haugeard, J.E., Cord, M., Thome, 391
392 N.: Jafar: Jack up any feature at any resolution (2025), [https://arxiv.](https://arxiv.org/abs/2506.11136) 392
393 [org/abs/2506.11136](https://arxiv.org/abs/2506.11136) 393
394 [9] Darcet, T., Oquab, M., Mairal, J., Bojanowski, P.: Vision transformers need 394
395 registers (2024), <https://arxiv.org/abs/2309.16588> 395
396 [10] Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A 396
397 large-scale hierarchical image database. In: 2009 IEEE conference on com- 397
398 puter vision and pattern recognition. pp. 248–255. Ieee (2009) 398
399 [11] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Un- 399
400 terthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, 400
401 J., Houlsby, N.: An image is worth 16x16 words: Transformers for image 401
402 recognition at scale (2021), <https://arxiv.org/abs/2010.11929> 402
403 [12] Fu, S., Hamilton, M., Brandt, L.E., Feldmann, A., Zhang, Z., Freeman, 403
404 W.T.: Featup: A model-agnostic framework for features at any resolu- 404
405 tion. In: The Twelfth International Conference on Learning Representations 405
406 (2024), <https://openreview.net/forum?id=GkJiNn2QDF> 406
407 [13] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recog- 407
408 nition (2015), <https://arxiv.org/abs/1512.03385> 408

- [14] Huang, H., Chen, A., Havrylov, V., Geiger, A., Zhang, D.: Loftup: Learning a coordinate-based feature upsampler for vision foundation models. arXiv preprint arXiv:2504.14032 (2025)
- [15] Jain, J., Li, J., Chiu, M., Hassani, A., Orlov, N., Shi, H.: Oneformer: One transformer to rule universal image segmentation (2022), <https://arxiv.org/abs/2211.06220>
- [16] Lin, T.Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P.: Microsoft coco: Common objects in context (2015), <https://arxiv.org/abs/1405.0312>
- [17] Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Jiang, Q., Li, C., Yang, J., Su, H., Zhu, J., Zhang, L.: Grounding dino: Marrying dino with grounded pre-training for open-set object detection (2024), <https://arxiv.org/abs/2303.05499>
- [18] Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al.: DINOv2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193 (2023)
- [19] Peebles, W., Xie, S.: Scalable diffusion models with transformers (2023), <https://arxiv.org/abs/2212.09748>
- [20] Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sasstry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning transferable visual models from natural language supervision (2021), <https://arxiv.org/abs/2103.00020>
- [21] Ranftl, R., Bochkovskiy, A., Koltun, V.: Vision transformers for dense prediction (2021), <https://arxiv.org/abs/2103.13413>
- [22] Ranftl, R., Lasinger, K., Hafner, D., Schindler, K., Koltun, V.: Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer (2020), <https://arxiv.org/abs/1907.01341>
- [23] Roberts, M., Ramapuram, J., Ranjan, A., Kumar, A., Bautista, M.A., Paczan, N., Webb, R., Susskind, J.M.: Hypersim: A photorealistic synthetic dataset for holistic indoor scene understanding (2021), <https://arxiv.org/abs/2011.02523>
- [24] Siméoni, O., Vo, H.V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., Darcet, T., Moutakanni, T., Sentana, L., Roberts, C., Vedaldi, A., Tolan, J., Brandt, J., Couprie, C., Mairal, J., Jégou, H., Labatut, P., Bojanowski, P.: DINOv3 (2025), <https://arxiv.org/abs/2508.10104>
- [25] Suri, S., Walmer, M., Gupta, K., Shrivastava, A.: Lift: A surprisingly simple lightweight feature transform for dense vit descriptors (2024), <https://arxiv.org/abs/2403.14625>
- [26] Tumanyan, N., Singer, A., Bagon, S., Dekel, T.: Dino-tracker: Taming dino for self-supervised point tracking in a single video (2024), <https://arxiv.org/abs/2403.14548>
- [27] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), <https://arxiv.org/abs/1706.03762>

- [28] Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing* **13**(4), 600–612 (2004)
- [29] Wang, Z., Yu, J., Yu, A.W., Dai, Z., Tsvetkov, Y., Cao, Y.: Simvln: Simple visual language model pretraining with weak supervision (2022), <https://arxiv.org/abs/2108.10904>
- [30] Wimmer, T., Truong, P., Rakotosaona, M.J., Oechsle, M., Tombari, F., Schiele, B., Lenssen, J.E.: Anyup: Universal feature upsampling. *arXiv preprint arXiv:2510.12764* (2025)
- [31] Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., Yang, J.: Structured 3d latents for scalable and versatile 3d generation (2025), <https://arxiv.org/abs/2412.01506>
- [32] Yang, J., Luo, K.Z., Li, J., Deng, C., Guibas, L., Krishnan, D., Weinberger, K.Q., Tian, Y., Wang, Y.: Denoising vision transformers (2024), <https://arxiv.org/abs/2401.02957>
- [33] Yang, L., Kang, B., Huang, Z., Xu, X., Feng, J., Zhao, H.: Depth anything: Unleashing the power of large-scale unlabeled data (2024), <https://arxiv.org/abs/2401.10891>
- [34] Yang, L., Kang, B., Huang, Z., Zhao, Z., Xu, X., Feng, J., Zhao, H.: Depth anything v2 (2024), <https://arxiv.org/abs/2406.09414>
- [35] Yun, S., Lee, H., Kim, J., Shin, J.: Patch-level representation learning for self-supervised vision transformers (2022), <https://arxiv.org/abs/2206.07990>
- [36] Zhai, X., Kolesnikov, A., Houlsby, N., Beyer, L.: Scaling vision transformers (2022), <https://arxiv.org/abs/2106.04560>
- [37] Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A., Torralba, A.: Scene parsing through ade20k dataset. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2017)
- [38] Zhou, B., Zhao, H., Puig, X., Xiao, T., Fidler, S., Barriuso, A., Torralba, A.: Semantic understanding of scenes through ade20k dataset. *International Journal of Computer Vision (IJCV)* (2018)